Smart Document Analysis Workbench

The Smart Document Analysis Workbench allows the creation / editing of templates. It also includes an Admin Pannel for managing dynamic categories and drop-downs of the “template Studio” – The admin Panel is described in a separate document.

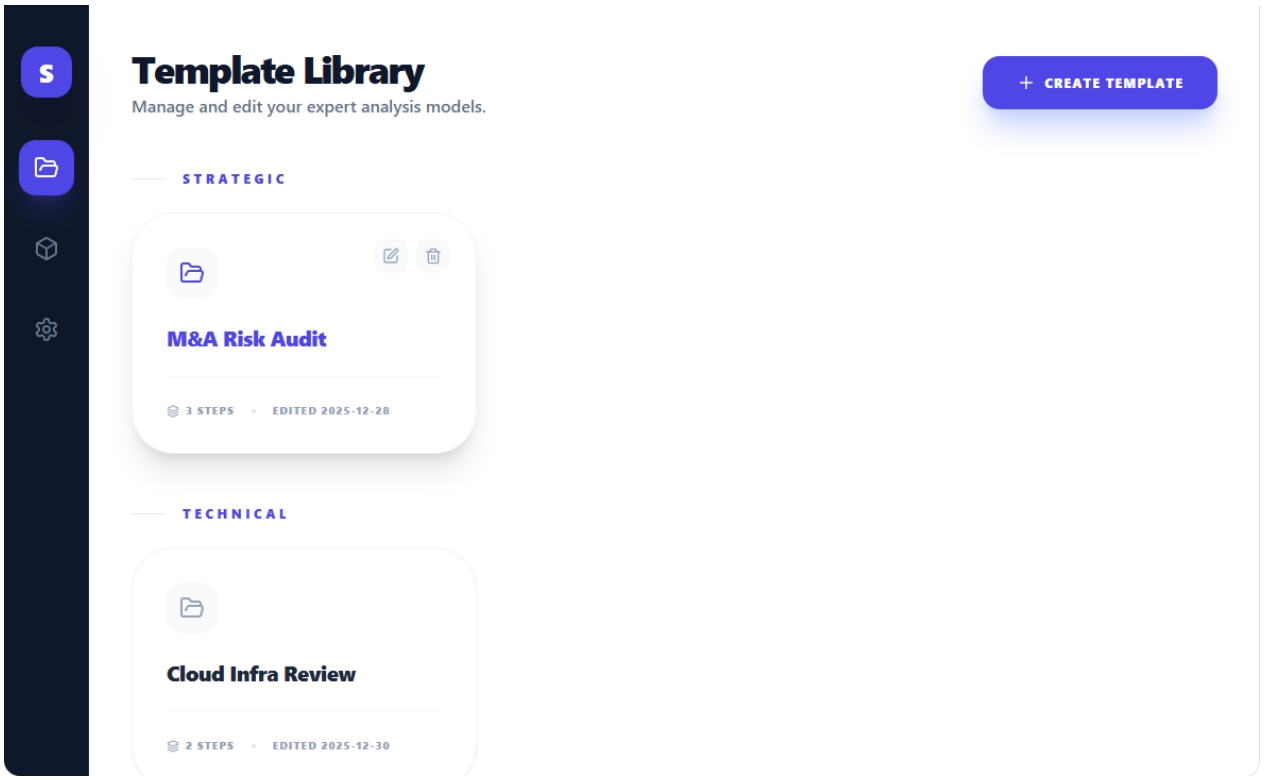

Figure 1 - Template Libary - Creating a new Template, Editing a template ("pencil" icon when you hover over a template) or delete a template ("Trash" icon when you hover over a template - requires user confirmation)

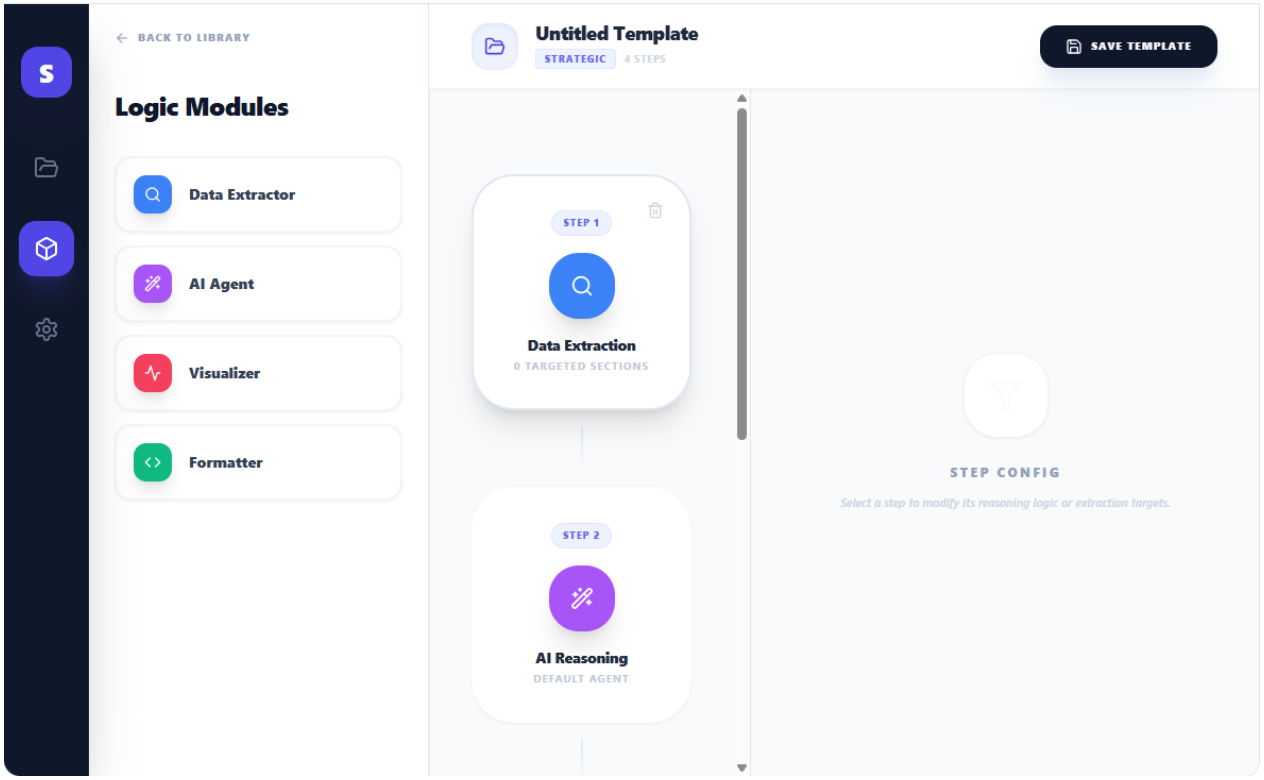

Figure 2: The Template Studio - Clicking on a Logic Module, adds it in the center pane. Hovering the mouse over one of the modules in the center pane displays the "Trash" icon to remove the module.

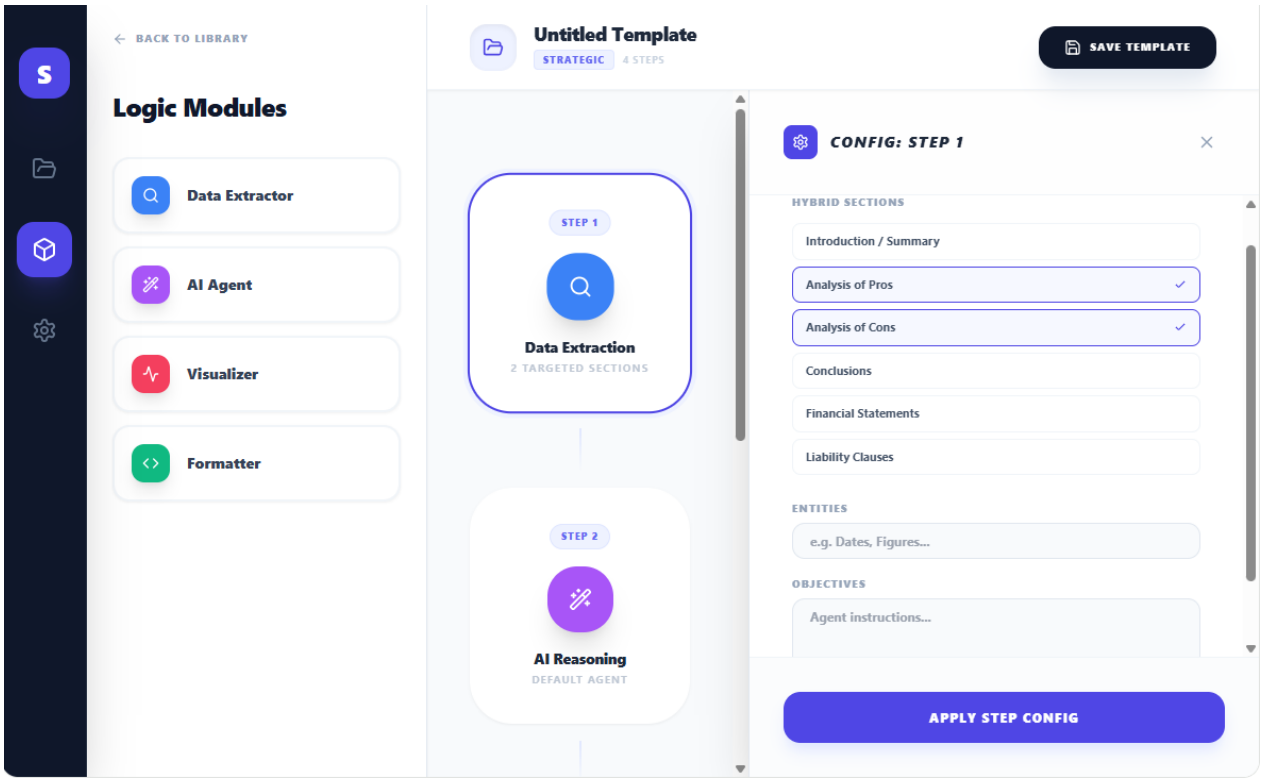

Figure 3: Data Extractor Module configuration - The user can select the sections that are targeted for the extraction, he can define the entities that must be extracted and provide the objective of the extraction to help the LLM for this activity

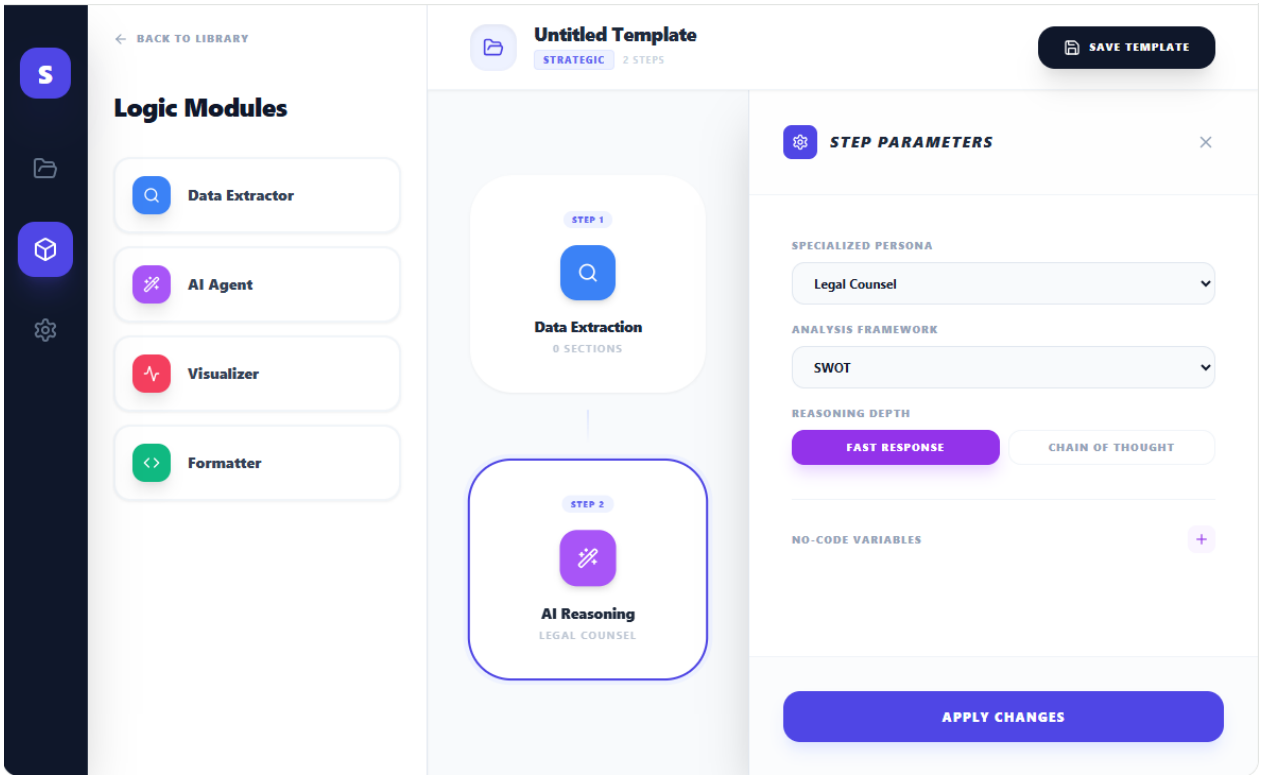

Figure 4: AI reasoning - The user can Define the persona of the AI that will do the analysis, What type of standard framework to be used (TOGAF, etc.), reasoning depth and variables that will have to be entered by the user upon applying the template

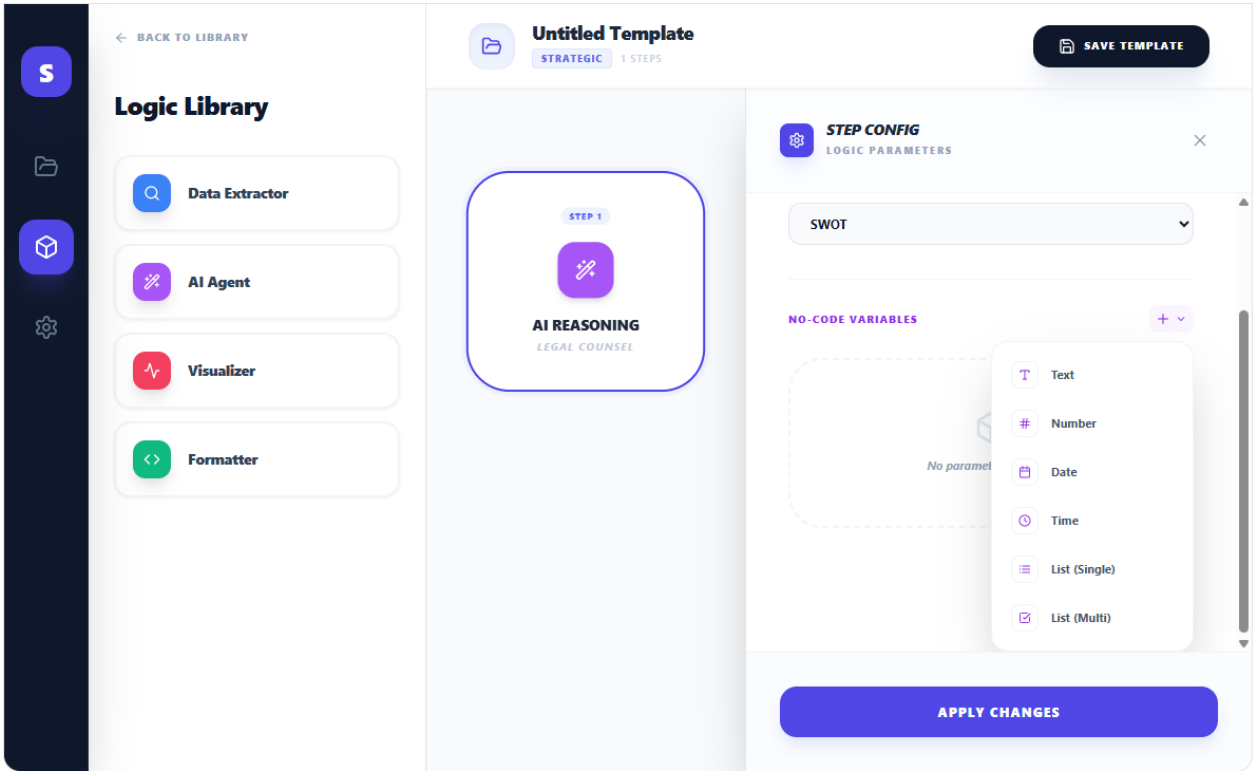

Figure 5: The designer can decide to create variables that will have to be entered by the user when he applies the template. The variables can be used by the AI when building its analysis. They can be of different types: "Text", "Number", "Date", "Time", "List (Single)" (The designer will have to define the list of items and the user will be allowed to select only one item in the list), "List (Multi)" (The designer will have to define the list of items and the user will be allowed to select several items in the list)

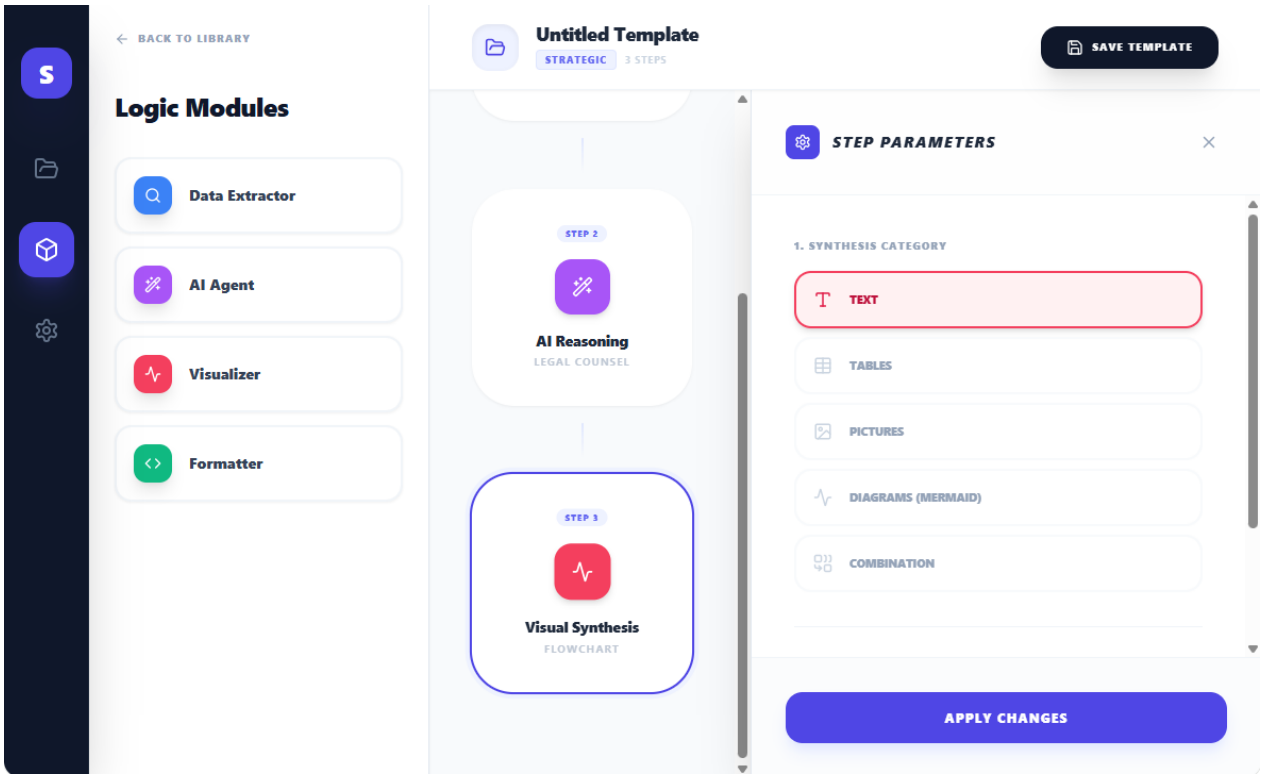

Figure 6: Visual Synthesis - The user can select the type of output that he wants (Text, Table, etc.)

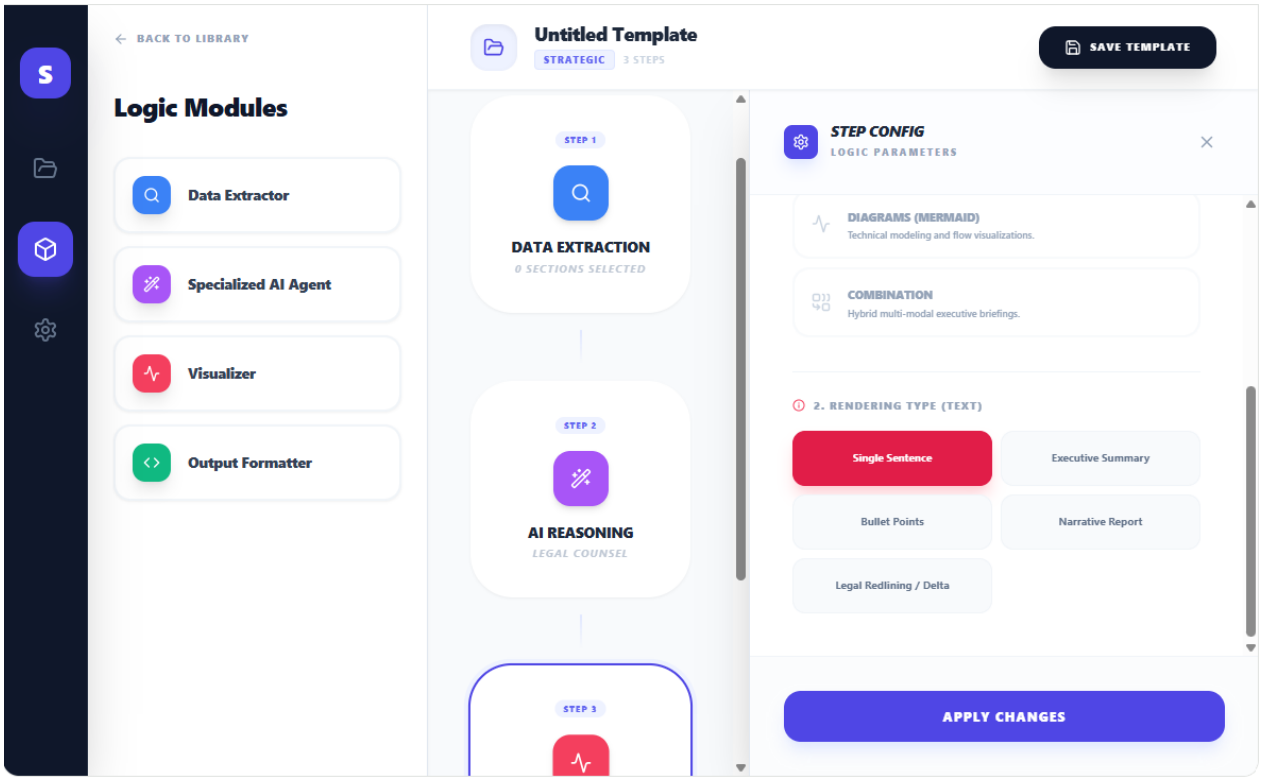

Figure 7: Visual Synthesis - The user can select the type of rendering - For text there are “Single Sentence”, “Executive Summary”, “Bullet Points”, “Narrative Report” and “Legal Redlining / Delta”

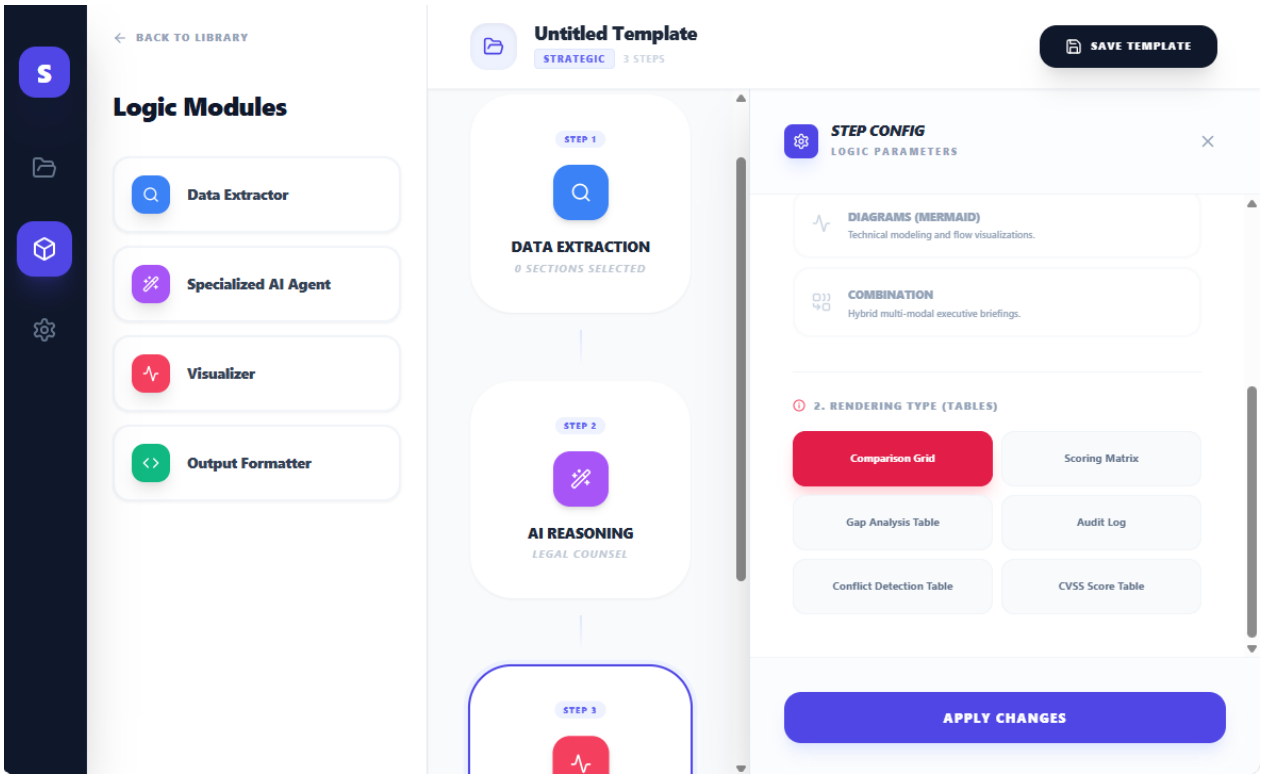

Figure 8: Visual Synthesis - The user can select the type of rendering - For Tables, these are "Comparison Grid", "Gap Analysis Table", "Conflict Detection Table", "Scoring Matrix", "Audit Log", "CVSS Score Table"

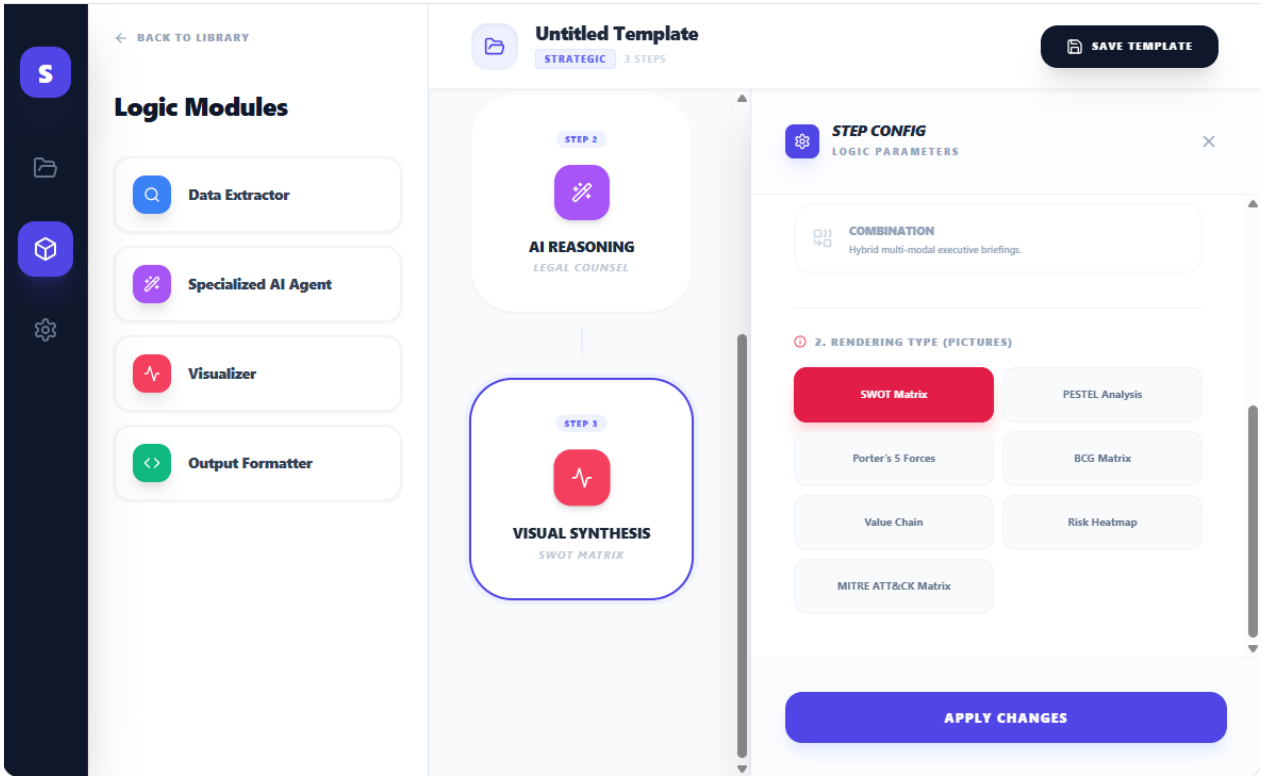

Figure 9: Visual Synthesis - The user can select the type of rendering - For Pictures we have “SWOT Matrix”, “PESTEL Analysis”, “Porter’s 5 Forces”, “BCG Matrix”, “Value Chain”, “Risk Heatmap”, “MITRE ATT&CK Matrix”

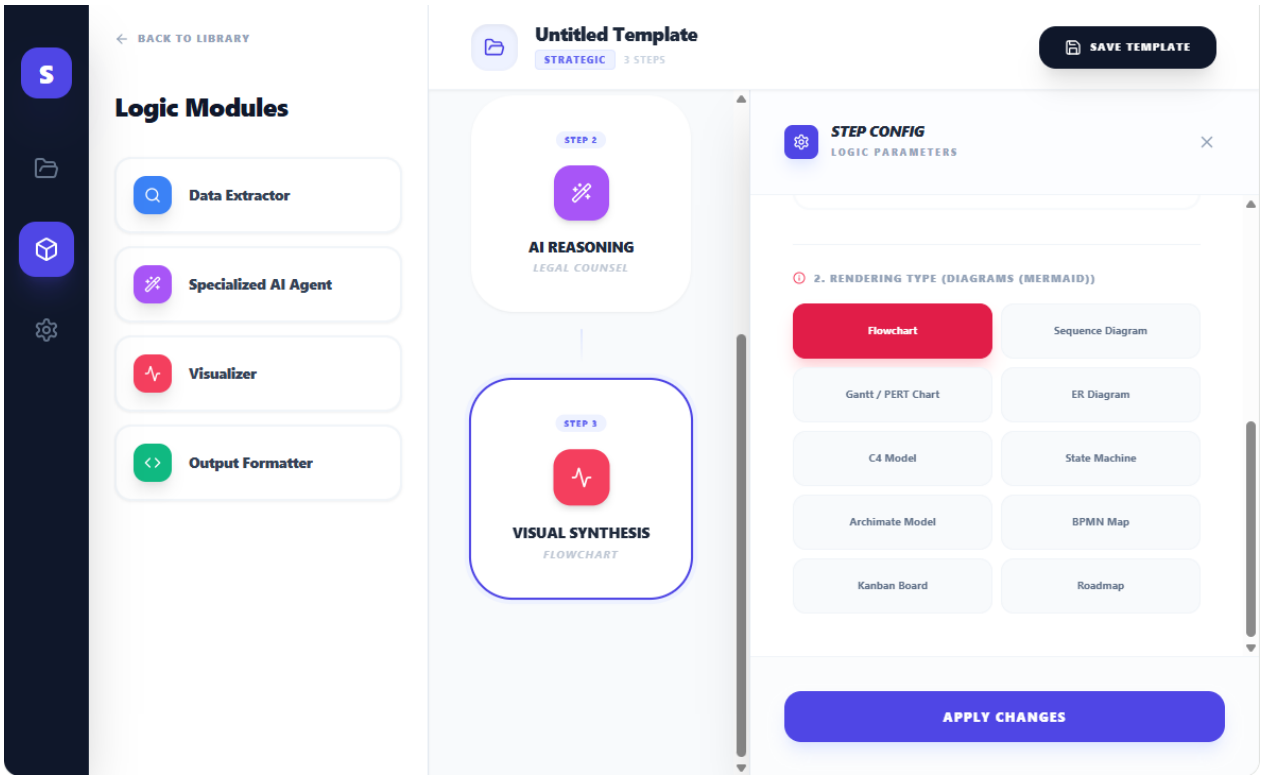

Figure 10: Visual Synthesis - The user can select the type of rendering - For Diagrams these are "Flowchart", "Gantt / PERT Chart", "C4 Model", "Archimate Model", "Kanban Board", "Sequence Diagram", "ER Diagram", "State Machine", "BPMN Map", "Roadmap"

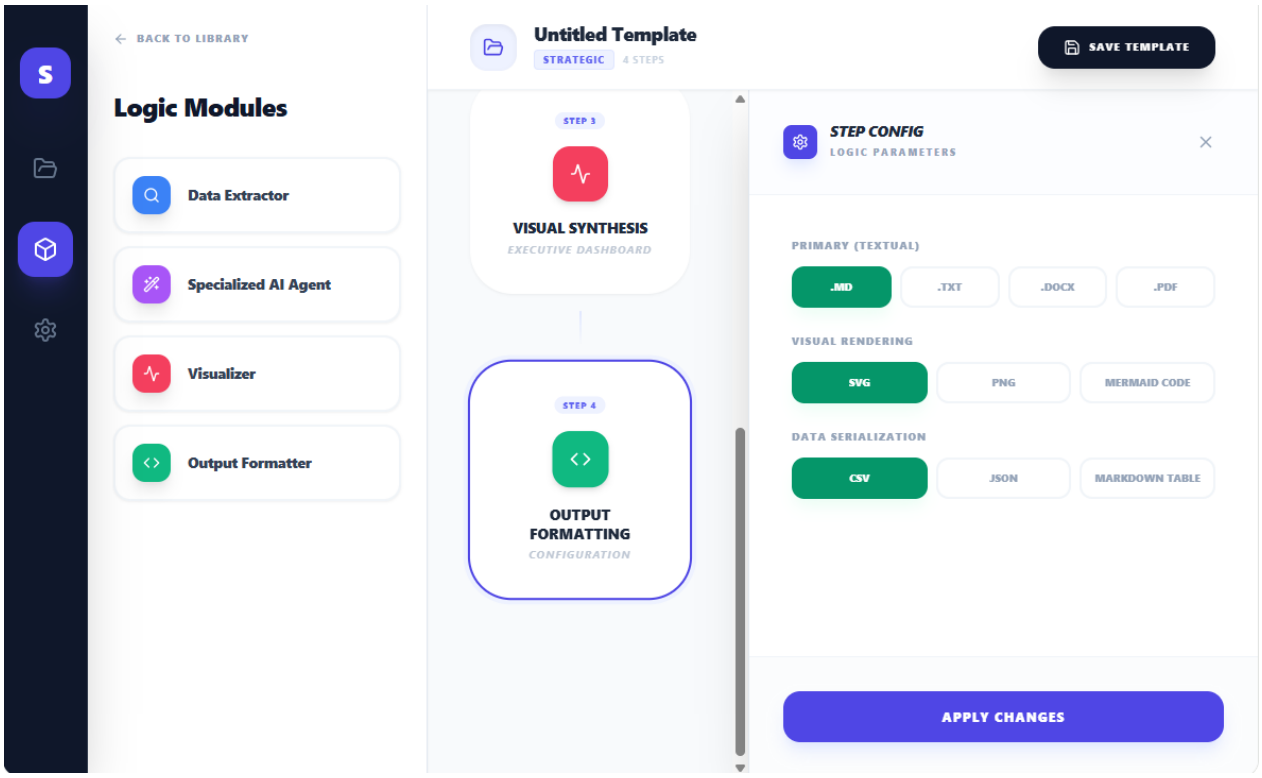

Figure 11: Finally, for Output Formatter, The user can select the format of the Output, based on the type of rendering

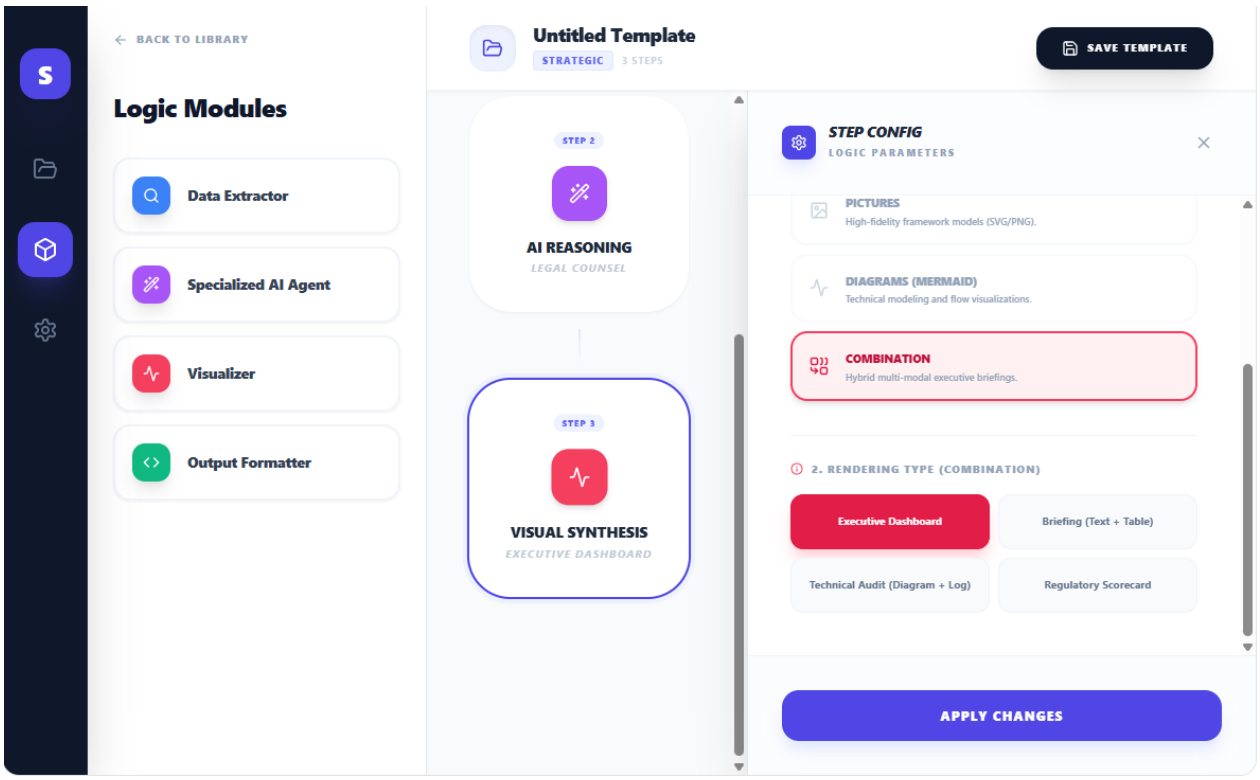

Figure 12: Visual Synthesis - The user can select the type of rendering - For Combinations these are "Executive Dashboard", "Technical Audit (Diagram + Log)", "Briefing (Text + Table)", "Regulatory Scoreboard"

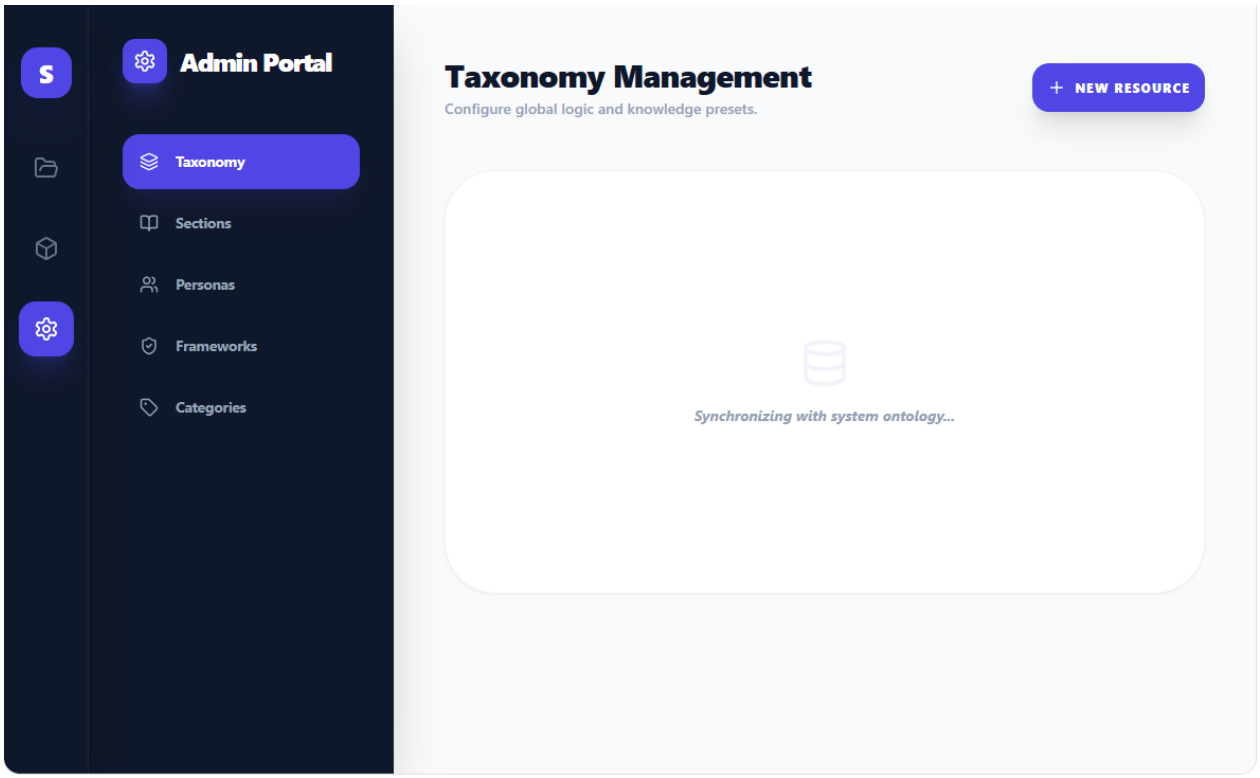

Figure 13: The last option (The "gear") is for the Template Builder. It is described in details in a separate document

Code for the Mockup

Here is the code of the Mockup for this Visual Workbench:

```
import React, { useState, useMemo } from 'react';

import {

  Settings, Play, ChevronRight, Search, Layers,

  Zap, CheckCircle2, Plus, LayoutDashboard,

  Trash2, Save, Wand2, Box, Terminal,

  Activity, Tag, Globe, Users, X, Check, Filter, HelpCircle,

  BookOpen, List, Type, Calendar, Hash, FolderOpen, ArrowLeft, MoreVertical, Edit,

  Code, Database, FileText, ShieldCheck, Layout, Table as TableIcon, Image as ImageIcon, Combine,

  BarChart, PieChart, Info, Map, ChevronDown, Clock, CheckSquare

} from 'lucide-react';


// --- INITIAL MOCK DATA ---
```

```
const INITIAL_CATEGORIES = [
  { id: 'cat_strat', name: 'Strategic', context: 'Taxonomy' },
  { id: 'cat_tech', name: 'Technical', context: 'Taxonomy' },
  { id: 'cat_comp', name: 'Compliance', context: 'Taxonomy' },
];

const INITIAL_SECTIONS = [
  { id: 's1', name: 'Introduction / Summary', group: 'Generic' },
  { id: 's2', name: 'Analysis of Pros', group: 'Generic' },
  { id: 's3', name: 'Analysis of Cons', group: 'Generic' },
  { id: 's4', name: 'Conclusions', group: 'Generic' },
  { id: 's5', name: 'Financial Statements', group: 'Domain-Specific', category: 'Financial' },
  { id: 's6', name: 'Liability Clauses', group: 'Domain-Specific', category: 'Legal' },
];

const INITIAL_PERSONAS = [
  { id: 'p1', name: 'Legal Counsel' },
  { id: 'p2', name: 'Financial Auditor' },
  { id: 'p3', name: 'Cybersecurity Architect' },
  { id: 'p4', name: 'Strategic Consultant' },
];

const INITIAL_FRAMEWORKS = [
  { id: 'f1', name: 'SWOT', category: 'Strategic' },
  { id: 'f2', name: 'STRIDE', category: 'Risk' },
  { id: 'f3', name: 'TOGAF', category: 'Technical' },
  { id: 'f4', name: 'PESTEL', category: 'Strategic' },
  { id: 'f5', name: 'NIST CSF', category: 'Risk' },
];

const VISUALIZATION_CATALOG = {
  'Text': {
    icon: <Type size={18} />,
    description: 'Summaries and narrative interpretations.',
    types: ['Single Sentence', 'Executive Summary', 'Bullet Points', 'Narrative Report', 'Legal Redlining / Delta']
```

```
  },
  'Tables': {
    icon: <TableIcon size={18} />,
    description: 'Structured analytical matrices and logs.',
    types: ['Comparison Grid', 'Scoring Matrix', 'Gap Analysis Table', 'Audit Log', 'Conflict Detection Table', 'CVSS Score Table']
  },
  'Pictures': {
    icon: <ImageIcon size={18} />,
    description: 'High-fidelity framework models (SVG/PNG).',
    types: ['SWOT Matrix', 'PESTEL Analysis', 'Porter’s 5 Forces', 'BCG Matrix', 'Value Chain', 'Risk Heatmap', 'MITRE ATT&CK Matrix']
  },
  'Diagrams (Mermaid)': {
    icon: <Activity size={18} />,
    description: 'Technical modeling and flow visualizations.',
    types: ['Flowchart', 'Sequence Diagram', 'Gantt / PERT Chart', 'ER Diagram', 'C4 Model', 'State Machine', 'Archimate Model', 'BPMN Map', 'Kanban Board', 'Roadmap']
  },
  'Combination': {
    icon: <Combine size={18} />,
    description: 'Hybrid multi-modal executive briefings.',
    types: ['Executive Dashboard', 'Briefing (Text + Table)', 'Technical Audit (Diagram + Log)', 'Regulatory Scorecard']
  }
};

const INITIAL_TEMPLATES = [
  { id: 't1', name: 'M&A Risk Audit', category: 'Strategic', steps: 3, lastEdited: '2025-12-28' },
  { id: 't2', name: 'ISO 27001 Compliance', category: 'Compliance', steps: 4, lastEdited: '2025-12-29' },
  { id: 't3', name: 'Cloud Infra Review', category: 'Technical', steps: 2, lastEdited: '2025-12-30' },
];

const App = () => {
  const [view, setView] = useState('browser');
```

```
  const [templates, setTemplates] = useState(INITIAL_TEMPLATES);
  const [activeTemplate, setActiveTemplate] = useState(null);
  const [isAnalyzing, setIsAnalyzing] = useState(false);
  const [showResult, setShowResult] = useState(false);

  const [pipeline, setPipeline] = useState([]);
  const [activeConfigBlock, setActiveConfigBlock] = useState(null);
  const [showSaveModal, setShowSaveModal] = useState(false);
  const [saveFormData, setSaveFormData] = useState({ name: '', category: 'Strategic' });
  const [adminTab, setAdminTab] = useState('taxonomy');

  const handleCreateNew = () => {
    setActiveTemplate(null);
    setPipeline([
      { id: 'b1', type: 'extract', config: { sections: [], entities: '', objectives: '' } }
    ]);
    setView('studio');
    setSaveFormData({ name: 'New Template', category: 'Strategic' });
  };

  const handleEditTemplate = (tpl) => {
    setActiveTemplate(tpl);
    setSaveFormData({ name: tpl.name, category: tpl.category });
    setPipeline([
      { id: 'b1', type: 'extract', config: { sections: ['Introduction / Summary'], entities:
'Risks', objectives: '' } },
      { id: 'b2', type: 'ai_agent', config: { persona: 'Legal Counsel', framework: 'SWOT',
logicDepth: 'Chain of Thought', variables: [] } },
      { id: 'b3', type: 'visual', config: { visCategory: 'Diagrams (Mermaid)', visType:
'Flowchart' } }
    ]);
    setView('studio');
  };

  const handleDeleteTemplate = (id) => {
    setTemplates(templates.filter(t => t.id !== id));
```

```
  };

  const handleSaveTemplate = () => {
    if (activeTemplate) {
      setTemplates(templates.map(t => t.id === activeTemplate.id ? { ...t, name: saveFormData.name, category: saveFormData.category } : t));
    } else {
      const newTpl = {
        id: Math.random().toString(),
        name: saveFormData.name,
        category: saveFormData.category,
        steps: pipeline.length,
        lastEdited: new Date().toISOString().split('T')[0]
      };
      setTemplates([newTpl, ...templates]);
    }
    setShowSaveModal(false);
    setView('browser');
  };

  const addBlock = (type) => {
    const newBlock = {
      id: Math.random().toString(),
      type,
      config: type === 'ai_agent' ? { variables: [], persona: 'Legal Counsel', framework: 'SWOT', logicDepth: 'Fast Response' } :
              type === 'visual' ? { visCategory: 'Text', visType: 'Executive Summary' } :
              type === 'format' ? { textFormat: '.md', visualFormat: 'SVG', dataFormat: 'CSV' } :
              { sections: [], entities: '', objectives: '' }
    };
    setPipeline([...pipeline, newBlock]);
  };

  const updateBlockConfig = (id, newConfig) => {
    setPipeline(p => p.map(b => b.id === id ? { ...b, config: { ...b.config, ...newConfig } } : b));
```

```
  };


  // --- SUB-FORMS ---


  const DataExtractorForm = ({ block }) => (

    <div className="space-y-6 animate-in fade-in slide-in-from-right-4 duration-300">

      <div>

        <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest mb-3
block">Hybrid Section Extraction</label>

        <div className="grid grid-cols-1 gap-1.5">

          {INITIAL_SECTIONS.map(s => (

            <div

              key={s.id}

              onClick={() => {

                const current = block.config.sections || [];

                const next = current.includes(s.name) ? current.filter(x => x !== s.name) :
[...current, s.name];

                updateBlockConfig(block.id, { sections: next });

              }}

              className={`px-4 py-2.5 rounded-xl border-2 cursor-pointer transition-all flex
items-center justify-between ${block.config.sections?.includes(s.name) ? 'border-indigo-600 bg-
indigo-50/50' : 'border-slate-100 bg-white'}`}

            >

              <div className="flex flex-col">

                <span className="text-[11px] font-bold text-slate-700">{s.name}</span>

                <span className="text-[8px] text-slate-400 font-bold uppercase tracking-
wider">{s.group}</span>

              </div>

              {block.config.sections?.includes(s.name) && <Check size={14} className="text-
indigo-600" strokeWidth={3} />}

            </div>

          ))}

        </div>

      </div>

      <div className="space-y-4 pt-4 border-t border-slate-50">

        <div>

          <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest mb-
1.5 block text-indigo-600">Specific Entities</label>
```

```
          <input

            type="text"

            className="w-full bg-slate-50 border border-slate-200 rounded-xl px-4 py-2 text-xs
font-bold outline-none focus:border-indigo-300"

            placeholder="e.g. Liability limits, Termination clauses..."

            defaultValue={block.config.entities}

            onChange={(e) => updateBlockConfig(block.id, { entities: e.target.value })}

          />

        </div>

        <div>

          <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest mb-
1.5 block text-indigo-600">Custom Objectives</label>

          <textarea

            className="w-full bg-slate-50 border border-slate-200 rounded-xl px-4 py-2 text-xs
font-bold outline-none focus:border-indigo-300 h-24"

            placeholder="What specific outcome should the parser achieve?"

            defaultValue={block.config.objectives}

            onChange={(e) => updateBlockConfig(block.id, { objectives: e.target.value })}

          />

        </div>

      </div>

    </div>

  );


  const AiAgentForm = ({ block }) => {

    const varTypes = [

      { id: 'text', label: 'Text', icon: <Type size={12} /> },

      { id: 'number', label: 'Number', icon: <Hash size={12} /> },

      { id: 'date', label: 'Date', icon: <Calendar size={12} /> },

      { id: 'time', label: 'Time', icon: <Clock size={12} /> },

      { id: 'list_single', label: 'List (Single)', icon: <List size={12} /> },

      { id: 'list_multi', label: 'List (Multi)', icon: <CheckSquare size={12} /> },

    ];


    const addVar = (type) => {

      const current = block.config.variables || [];
```

```
      const newVar = {
        id: Date.now(),
        label: 'Untitled Variable',
        type: type,
        description: '',
        options: type.includes('list') ? 'Option A, Option B' : null
      };
      updateBlockConfig(block.id, { variables: [...current, newVar] });
    };

    return (
      <div className="space-y-8 animate-in fade-in slide-in-from-right-4 duration-300">
        <div className="space-y-4">
          <div>
            <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest mb-
2 block">Agent Persona</label>
            <select
              className="w-full bg-slate-50 border border-slate-200 rounded-xl px-4 py-2.5 text-
xs font-bold outline-none cursor-pointer"
              defaultValue={block.config.persona}
              onChange={(e) => updateBlockConfig(block.id, { persona: e.target.value })}
            >
              {INITIAL_PERSONAS.map(p => <option key={p.id} value={p.name}>{p.name}</option>)}
            </select>
          </div>
          <div>
            <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest mb-
2 block">Logical Framework</label>
            <select
              className="w-full bg-slate-50 border border-slate-200 rounded-xl px-4 py-2.5 text-
xs font-bold outline-none cursor-pointer"
              defaultValue={block.config.framework}
              onChange={(e) => updateBlockConfig(block.id, { framework: e.target.value })}
            >
              {INITIAL_FRAMEWORKS.map(f => <option key={f.id} value={f.name}>{f.name}</option>)}
            </select>
          </div>
```

```
        </div>


        <div className="pt-6 border-t border-slate-100">

          <div className="flex justify-between items-center mb-6">

            <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest
block text-purple-600">No-Code Variables</label>

            <div className="relative group">

               <button className="p-1.5 bg-purple-50 text-purple-600 rounded-lg hover:bg-purple-
600 hover:text-white transition-all flex items-center gap-1">

                 <Plus size={14}/>

                 <ChevronDown size={12}/>

               </button>

               <div className="absolute right-0 top-full mt-2 w-48 bg-white border border-slate-
100 shadow-2xl rounded-2xl p-2 z-[100] hidden group-hover:block animate-in zoom-in-95 duration-
100">

                  {varTypes.map(vt => (

                    <button

                      key={vt.id}

                      onClick={() => addVar(vt.id)}

                      className="w-full flex items-center gap-3 p-2.5 hover:bg-purple-50 rounded-
xl text-left transition-colors"

                    >

                       <div className="text-purple-600 bg-white p-1.5 rounded-lg border border-
slate-100">{vt.icon}</div>

                       <span className="text-[11px] font-bold text-slate-600">{vt.label}</span>

                    </button>

                  ))}

               </div>

            </div>

          </div>


          <div className="space-y-6">

            {(block.config.variables || []).map(v => (

              <div key={v.id} className="p-4 bg-slate-50/80 rounded-2xl border border-slate-100
space-y-4 group">

                <div className="flex items-center gap-3">

                  <div className="shrink-0 p-2 bg-white rounded-xl border border-slate-100 text-
purple-600 shadow-sm">

                    {varTypes.find(t => t.id === v.type)?.icon}
```

```
                  </div>
                  <input
                    type="text"
                    className="flex-1 bg-transparent border-none outline-none text-xs font-black
text-slate-700 min-w-0"
                    defaultValue={v.label}
                    placeholder="Variable Label..."
                    onBlur={(e) => {
                      const next = block.config.variables.map(x => x.id === v.id ? {...x, label:
e.target.value} : x);
                      updateBlockConfig(block.id, { variables: next });
                    }}
                  />
                  <button
                    onClick={() => updateBlockConfig(block.id, { variables:
block.config.variables.filter(x => x.id !== v.id) })}
                    className="text-slate-300 hover:text-rose-500 transition-colors"
                  >
                    <Trash2 size={14}/>
                  </button>
                </div>

                <div>
                  <div className="flex items-center gap-1 mb-1">
                    <label className="text-[9px] font-black text-slate-400 uppercase tracking-
widest">Mandatory Description</label>
                    <span className="text-rose-500 font-black text-[10px]">*</span>
                  </div>
                  <textarea
                    className="w-full bg-white border border-slate-200 rounded-xl px-3 py-2 text-
[11px] font-medium outline-none focus:border-purple-300 transition-all placeholder:text-slate-
300"
                    placeholder="Explain why the AI needs this value..."
                    defaultValue={v.description}
                    onBlur={(e) => {
                      const next = block.config.variables.map(x => x.id === v.id ? {...x,
description: e.target.value} : x);
                      updateBlockConfig(block.id, { variables: next });
```

```
                    }}
                  />
                </div>

                {v.type.includes('list') && (
                  <div>
                    <label className="text-[9px] font-black text-slate-400 uppercase tracking-widest mb-1 block">List Options (Comma separated)</label>
                    <input
                      type="text"
                      className="w-full bg-white border border-slate-200 rounded-xl px-3 py-2 text-[11px] font-bold outline-none focus:border-purple-300"
                      defaultValue={v.options}
                      onBlur={(e) => {
                        const next = block.config.variables.map(x => x.id === v.id ? {...x, options: e.target.value} : x);
                        updateBlockConfig(block.id, { variables: next });
                      }}
                    />
                  </div>
                )}
              </div>
            ))}
            {(!block.config.variables || block.config.variables.length === 0) && (
              <div className="py-12 border-2 border-dashed border-slate-100 rounded-[2rem] text-center">
                <Box size={32} className="mx-auto text-slate-200 mb-3" />
                <p className="text-[11px] text-slate-400 font-bold italic">No parameters defined.</p>
              </div>
            )}
          </div>
        </div>
      </div>
    );
  };
```

```
  const VisualizerForm = ({ block }) => {

    const selectedCat = block.config.visCategory || 'Text';

    const subTypes = VISUALIZATION_CATALOG[selectedCat]?.types || [];


    return (

      <div className="space-y-8 animate-in fade-in slide-in-from-right-4 duration-300">

        <div>

          <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest block
mb-4">1. Synthesis Category</label>

          <div className="grid grid-cols-1 gap-2">

            {Object.keys(VISUALIZATION_CATALOG).map(cat => (

              <button

                key={cat}

                onClick={() => updateBlockConfig(block.id, { visCategory: cat, visType:
VISUALIZATION_CATALOG[cat].types[0] })}

                className={`p-4 rounded-2xl border-2 text-left transition-all flex items-center
gap-4 ${block.config.visCategory === cat ? 'border-rose-500 bg-rose-50 text-rose-700 shadow-md' :
'border-slate-50 bg-white hover:border-rose-100 text-slate-400'}`}

              >

                <div className={`${block.config.visCategory === cat ? 'text-rose-600' : 'text-
slate-300'}`}>

                  {VISUALIZATION_CATALOG[cat].icon}

                </div>

                <div className="flex-1 min-w-0">

                  <span className="text-[11px] font-black uppercase tracking-tight
block">{cat}</span>

                  <p className="text-[9px] font-medium text-slate-400 leading-none mt-
1">{VISUALIZATION_CATALOG[cat].description}</p>

                </div>

              </button>

            ))}

          </div>

        </div>


        <div className="pt-6 border-t border-slate-100 animate-in fade-in slide-in-from-bottom-
2">

          <div className="flex items-center gap-2 mb-4">

             <Info size={12} className="text-rose-500" />
```

```
            <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest
block">2. Rendering Type ({selectedCat})</label>

          </div>

          <div className="grid grid-cols-2 gap-2">

            {subTypes.map(type => (

              <button

                key={type}

                onClick={() => updateBlockConfig(block.id, { visType: type })}

                className={`p-3 rounded-xl border text-[10px] font-bold transition-all text-
center leading-tight min-h-[52px] flex items-center justify-center ${block.config.visType ===
type ? 'bg-rose-600 border-rose-600 text-white shadow-lg shadow-rose-200' : 'bg-slate-50 border-
slate-100 text-slate-500 hover:bg-white hover:border-rose-300'}`}

              >

                {type}

              </button>

            ))}

          </div>

        </div>

      </div>

    );

  };


  const OutputFormatterForm = ({ block }) => (

    <div className="space-y-8 animate-in fade-in slide-in-from-right-4 duration-300">

      <div className="space-y-6">

        {[

          { label: 'Primary (Textual)', key: 'textFormat', options: ['.md', '.txt', '.docx',
'.pdf'] },

          { label: 'Visual Rendering', key: 'visualFormat', options: ['SVG', 'PNG', 'Mermaid
Code'] },

          { label: 'Data (Serialization)', key: 'dataFormat', options: ['CSV', 'JSON', 'Markdown
Table'] },

        ].map(group => (

          <div key={group.label}>

            <label className="text-[10px] font-black text-slate-400 uppercase tracking-widest mb-
3 block">{group.label}</label>

            <div className="flex gap-2">

              {group.options.map(opt => (

                <button
```

```
                  key={opt}
                  onClick={() => updateBlockConfig(block.id, { [group.key]: opt })}
                  className={`flex-1 py-2.5 rounded-xl text-[10px] font-black uppercase
transition-all border-2 ${block.config[group.key] === opt ? 'bg-emerald-600 border-emerald-600
text-white shadow-lg shadow-emerald-100' : 'bg-white border-slate-100 text-slate-400
hover:border-emerald-200'}`}
                >
                  {opt}
                </button>
              ))}
            </div>
          </div>
        ))}
      </div>
    </div>
  );

  // --- MAIN VIEWS ---

  const renderBrowser = () => (
    <div className="p-10 max-w-6xl mx-auto animate-in fade-in duration-500 h-full overflow-y-
auto">
      <header className="flex justify-between items-center mb-12">
        <div>
          <h1 className="text-4xl font-black tracking-tighter text-slate-900">Template
Library</h1>
          <p className="text-slate-500 font-medium mt-1">Manage and edit your expert analysis
models.</p>
        </div>
        <button
          onClick={handleCreateNew}
          className="bg-indigo-600 text-white px-8 py-4 rounded-2xl font-black text-xs uppercase
tracking-widest shadow-xl shadow-indigo-200 hover:bg-indigo-500 transition-all flex items-center
gap-2"
        >
          <Plus size={18} /> Create Template
        </button>
      </header>
```

```
      <div className="space-y-12 pb-20">
        {INITIAL_CATEGORIES.map(cat => {
          const catTemplates = templates.filter(t => t.category === cat.name);
          if (catTemplates.length === 0) return null;
          return (
            <div key={cat.id}>
              <h3 className="text-xs font-black text-indigo-600 uppercase tracking-[0.3em] mb-6
flex items-center gap-4">
                <div className="w-8 h-px bg-indigo-100" /> {cat.name}
              </h3>
              <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6">
                {catTemplates.map(tpl => (
                  <div key={tpl.id} className="group bg-white rounded-[2.5rem] p-8 border border-
slate-100 shadow-sm hover:shadow-xl transition-all relative overflow-hidden">
                    <div className="absolute top-0 right-0 p-6 opacity-0 group-hover:opacity-100
transition-all flex gap-2">
                      <button onClick={() => handleEditTemplate(tpl)} className="p-2 bg-slate-50
text-slate-400 hover:text-indigo-600 rounded-xl transition-colors"><Edit size={16}/></button>
                      <button onClick={() => handleDeleteTemplate(tpl.id)} className="p-2 bg-
slate-50 text-slate-400 hover:text-rose-600 rounded-xl transition-colors"><Trash2
size={16}/></button>
                    </div>
                    <div className="p-3 bg-slate-50 rounded-2xl w-fit mb-6 text-slate-400 group-
hover:text-indigo-600 transition-colors">
                      <FolderOpen size={24} />
                    </div>
                    <h4 className="text-xl font-black text-slate-800 tracking-tight leading-tight
mb-2 group-hover:text-indigo-600 transition-colors cursor-pointer" onClick={() =>
handleEditTemplate(tpl)}>
                      {tpl.name}
                    </h4>
                    <div className="flex items-center gap-4 mt-6 pt-6 border-t border-slate-50">
                      <div className="flex items-center gap-1.5 text-[10px] font-black text-
slate-400 uppercase tracking-widest">
                        <Layers size={12}/> {tpl.steps} Steps
                      </div>
                      <div className="w-1 h-1 bg-slate-200 rounded-full" />
                      <div className="text-[10px] font-black text-slate-400 uppercase tracking-
widest">
                        Edited {tpl.lastEdited}
```

```
                    </div>
                  </div>
                </div>
              ))}
            </div>
          </div>
        );
      })}
    </div>
  </div>
);


const renderStudio = () => (
  <div className="flex h-full bg-[#F8FAFC] overflow-hidden">
    <aside className="w-80 bg-white border-r border-slate-200 p-6 flex flex-col shrink-0 shadow-sm z-10">
      <button onClick={() => setView('browser')} className="flex items-center gap-2 text-slate-400 hover:text-indigo-600 transition-colors mb-10 text-[10px] font-black uppercase tracking-widest group">
        <ArrowLeft size={14} className="group-hover:-translate-x-1 transition-transform" /> Back to Library
      </button>
      <h2 className="text-2xl font-black tracking-tight mb-8 text-slate-900">Logic Library</h2>

      <div className="space-y-3">
        {[
          { id: 'extract', name: 'Data Extractor', icon: <Search />, color: 'bg-blue-500' },
          { id: 'ai_agent', name: 'AI Agent', icon: <Wand2 />, color: 'bg-purple-500' },
          { id: 'visual', name: 'Visualizer', icon: <Activity />, color: 'bg-rose-500' },
          { id: 'format', name: 'Formatter', icon: <Code />, color: 'bg-emerald-500' },
        ].map(module => (
          <button
            key={module.id}
            onClick={() => addBlock(module.id)}
            className="w-full p-4 bg-white border-2 border-slate-100 rounded-2xl hover:border-indigo-600 transition-all flex items-center gap-4 group shadow-sm text-left"
          >
```

```
              <div className={`shrink-0 p-2.5 rounded-xl text-white ${module.color} shadow-lg
shadow-current/20`}>

                {module.icon && React.cloneElement(module.icon, { size: 16 })}

              </div>

              <span className="font-black text-sm text-slate-700 tracking-
tight">{module.name}</span>

            </button>

          ))}

        </div>

      </aside>


      <div className="flex-1 flex flex-col overflow-hidden">

        <header className="h-20 bg-white border-b border-slate-100 px-10 flex items-center
justify-between shrink-0 z-20 shadow-sm">

          <div className="flex items-center gap-4">

            <div className="p-3 bg-indigo-50 rounded-2xl text-indigo-600 shadow-
inner"><FolderOpen size={20}/></div>

            <div>

              <h3 className="text-lg font-black tracking-tight text-slate-800 leading-
none">{saveFormData.name || 'Untitled Template'}</h3>

              <div className="flex items-center gap-2 mt-1.5">

                <span className="text-[9px] font-black text-indigo-500 bg-indigo-50 px-2 py-
0.5 rounded uppercase tracking-widest border border-indigo-100">{saveFormData.category}</span>

                <span className="text-[9px] font-bold text-slate-300 uppercase tracking-
widest">{pipeline.length} Steps</span>

              </div>

            </div>

          </div>

          <button

            onClick={() => setShowSaveModal(true)}

            className="flex items-center gap-2 px-6 py-3 bg-slate-900 text-white rounded-2xl
font-black text-[10px] uppercase tracking-widest shadow-xl shadow-slate-200 hover:bg-slate-800
transition-all active:scale-95"

          >

            <Save size={16}/> Save Template

          </button>

        </header>


        <div className="flex-1 flex overflow-hidden">
```

```
          <div className="flex-1 p-10 overflow-y-auto bg-slate-50/30">

             <div className="max-w-md mx-auto py-10 pb-32">

               {pipeline.map((block, idx) => (

                 <React.Fragment key={block.id}>

                   <div

                     onClick={() => setActiveConfigBlock(block)}

                     className={`group relative bg-white rounded-[2.5rem] p-8 border-2
transition-all cursor-pointer shadow-sm hover:shadow-xl ${activeConfigBlock?.id === block.id ?
'border-indigo-600 ring-4 ring-indigo-50 shadow-indigo-100 scale-[1.02]' : 'border-transparent
hover:border-slate-200'}`}

                   >

                     <div className="flex flex-col items-center text-center gap-4 relative">

                       <div className="absolute top-0 right-0 flex gap-2 opacity-0 group-
hover:opacity-100 transition-opacity">

                         <button onClick={(e) => { e.stopPropagation();
setPipeline(pipeline.filter(x => x.id !== block.id))}} className="p-1.5 text-slate-300
hover:text-rose-500 transition-colors"><Trash2 size={14}/></button>

                       </div>

                       <span className="text-[8px] font-black text-indigo-500 bg-indigo-50 px-2
py-0.5 rounded-full uppercase tracking-widest">STEP {idx + 1}</span>

                       <div className={`p-4 rounded-2xl text-white shadow-md ${block.type ===
'extract' ? 'bg-blue-500' : block.type === 'ai_agent' ? 'bg-purple-500' : block.type === 'visual'
? 'bg-rose-500' : 'bg-emerald-500'}`}>

                         {block.type === 'extract' ? <Search size={20}/> : block.type ===
'ai_agent' ? <Wand2 size={20}/> : block.type === 'visual' ? <Activity size={20}/> : <Code
size={20}/>}

                       </div>

                       <div className="space-y-1">

                         <h4 className="text-sm font-black text-slate-800 tracking-tight
leading-tight uppercase">

                           {block.type === 'extract' ? 'Data Extraction' : block.type ===
'ai_agent' ? 'AI Reasoning' : block.type === 'visual' ? 'Visual Synthesis' : 'Output Formatting'}

                         </h4>

                         <p className="text-[10px] font-bold text-slate-400 uppercase tracking-
widest opacity-60 italic">

                           {block.type === 'extract' ? `${block.config.sections?.length || 0}
Sections Selected` :

                            block.type === 'visual' ? `${block.config.visType || 'Rendering
Config'}` :

                            block.type === 'ai_agent' ? `${block.config.persona || 'Auditor'}`
: 'Configuration'}

                         </p>

                       </div>
```

```
                    </div>
                  </div>
                  {idx < pipeline.length - 1 && (
                    <div className="flex justify-center py-4">
                      <div className="w-0.5 h-8 bg-gradient-to-b from-indigo-100 to-transparent rounded-full" />
                    </div>
                  )}
                </React.Fragment>
              ))}
              {pipeline.length === 0 && (
                <div className="h-64 border-4 border-dashed border-slate-100 rounded-[3rem] flex flex-col items-center justify-center text-slate-300 gap-4">
                  <Zap size={40} className="opacity-10" />
                  <p className="font-black uppercase tracking-widest text-[10px]">Add a module from the sidebar</p>
                </div>
              )}
            </div>
          </div>

          {activeConfigBlock ? (
            <aside className="w-[480px] bg-white border-l border-slate-200 shadow-2xl flex flex-col z-10 animate-in slide-in-from-right-10 duration-300">
              <header className="p-8 border-b border-slate-100 flex justify-between items-center bg-[#FCFDFF]">
                <div className="flex items-center gap-3">
                  <div className="p-2 bg-indigo-600 rounded-lg text-white shadow-lg shadow-indigo-100"><Settings size={16}/></div>
                  <div>
                    <h3 className="text-sm font-black text-slate-800 uppercase italic tracking-tight leading-none">Step Config</h3>
                    <p className="text-[9px] font-black text-slate-400 tracking-[0.2em] mt-1.5 uppercase">Logic Parameters</p>
                  </div>
                </div>
                <button onClick={() => setActiveConfigBlock(null)} className="p-2 hover:bg-slate-100 rounded-xl transition-all text-slate-400 hover:text-slate-600"><X size={18}/></button>
              </header>
```

```
              <div className="flex-1 overflow-y-auto p-10">

                 {activeConfigBlock.type === 'extract' && <DataExtractorForm
block={activeConfigBlock} />}

                 {activeConfigBlock.type === 'ai_agent' && <AiAgentForm
block={activeConfigBlock} />}

                 {activeConfigBlock.type === 'visual' && <VisualizerForm
block={activeConfigBlock} />}

                 {activeConfigBlock.type === 'format' && <OutputFormatterForm
block={activeConfigBlock} />}

              </div>

              <footer className="p-8 border-t border-slate-100 bg-slate-50/50">

                 <button onClick={() => setActiveConfigBlock(null)} className="w-full py-4 bg-
indigo-600 text-white rounded-2xl font-black text-xs uppercase tracking-widest shadow-lg shadow-
indigo-100 hover:bg-indigo-50 transition-all">Apply Changes</button>

              </footer>

            </aside>

          ) : (

            <aside className="w-[480px] bg-slate-50/30 border-l border-slate-200 flex flex-col
items-center justify-center p-16 text-center text-slate-300 gap-6">

               <div className="w-20 h-20 bg-white rounded-[2.5rem] shadow-sm border border-
slate-100 flex items-center justify-center"><Filter size={32} className="opacity-10" /></div>

               <div>

                  <p className="font-black text-xs uppercase tracking-widest text-slate-
400">Contextual Editor</p>

                  <p className="text-[11px] font-medium mt-3 leading-relaxed italic max-w-xs mx-
auto">Select a pipeline step to configure its reasoning logic, extraction targets, or rendering
style.</p>

               </div>

            </aside>

          )}

        </div>

      </div>



      {showSaveModal && (

        <div className="fixed inset-0 z-[100] flex items-center justify-center p-6 animate-in
fade-in duration-200">

          <div className="absolute inset-0 bg-slate-900/60 backdrop-blur-sm" onClick={() =>
setShowSaveModal(false)} />

          <div className="relative bg-white w-full max-w-md rounded-[3rem] shadow-2xl p-10
overflow-hidden animate-in zoom-in-95 duration-200">

            <h3 className="text-2xl font-black tracking-tighter text-slate-900 mb-6">Finalize
Template</h3>
```

```
            <div className="space-y-6">
               <div>
                  <label className="text-[10px] font-black text-slate-400 uppercase tracking-
widest mb-2 block">Display Name</label>
                  <input
                    type="text"
                    className="w-full bg-slate-50 border border-slate-200 rounded-xl px-4 py-3
text-sm font-bold outline-none focus:border-indigo-300 transition-all"
                    value={saveFormData.name}
                    onChange={(e) => setSaveFormData({...saveFormData, name: e.target.value})}
                  />
               </div>
               <div>
                  <label className="text-[10px] font-black text-slate-400 uppercase tracking-
widest mb-2 block">Target Category</label>
                  <select
                    className="w-full bg-slate-50 border border-slate-200 rounded-xl px-4 py-3
text-sm font-bold outline-none cursor-pointer hover:bg-slate-100 transition-all"
                    value={saveFormData.category}
                    onChange={(e) => setSaveFormData({...saveFormData, category:
e.target.value})}
                  >
                    {INITIAL_CATEGORIES.map(c => <option key={c.id}
value={c.name}>{c.name}</option>)}
                  </select>
               </div>
            </div>
            <div className="flex gap-4 mt-10">
               <button onClick={() => setShowSaveModal(false)} className="flex-1 py-4 bg-slate-50
text-slate-500 rounded-2xl font-black text-xs uppercase tracking-widest hover:bg-slate-100
transition-all">Discard</button>
               <button onClick={handleSaveTemplate} className="flex-1 py-4 bg-indigo-600 text-
white rounded-2xl font-black text-xs uppercase tracking-widest shadow-xl shadow-indigo-100
hover:bg-indigo-50 transition-all">Confirm & Save</button>
            </div>
          </div>
        </div>
      )}
    </div>
```

```
  );


  const renderAdmin = () => (

    <div className="flex h-full bg-[#F8FAFC]">

      <aside className="w-72 bg-slate-900 p-8 flex flex-col shrink-0 shadow-2xl">

        <div className="flex items-center gap-3 mb-12">

          <div className="p-2.5 bg-indigo-600 rounded-xl text-white shadow-xl shadow-indigo-
600/30"><Settings size={22} /></div>

          <h2 className="text-2xl font-black text-white tracking-tighter">Admin Portal</h2>

        </div>

        <nav className="space-y-1.5 flex-1">

          {[

            { id: 'taxonomy', icon: <Layers />, label: 'Taxonomy' },

            { id: 'sections', icon: <BookOpen />, label: 'Sections' },

            { id: 'personas', icon: <Users />, label: 'Personas' },

            { id: 'frameworks', icon: <ShieldCheck />, label: 'Frameworks' },

            { id: 'categories', icon: <Tag />, label: 'Categories' },

          ].map(item => (

            <button

              key={item.id}

              onClick={() => setAdminTab(item.id)}

              className={`w-full flex items-center gap-4 p-4 rounded-2xl transition-all
${adminTab === item.id ? 'bg-indigo-600 text-white shadow-xl shadow-indigo-900/20' : 'text-slate-
400 hover:text-white hover:bg-slate-800'}`}

            >

              {item.icon && React.cloneElement(item.icon, { size: 18 })}

              <span className="text-sm font-bold tracking-tight">{item.label}</span>

            </button>

          ))}

        </nav>

      </aside>


      <main className="flex-1 p-12 overflow-y-auto">

        <header className="flex justify-between items-center mb-12">

           <div>

              <h2 className="text-3xl font-black tracking-tight capitalize text-slate-
900">{adminTab} Management</h2>
```

```
              <p className="text-slate-400 font-medium text-sm mt-1">Configure global logic and
knowledge presets.</p>

           </div>

           <button className="p-3.5 bg-indigo-600 text-white rounded-2xl shadow-xl hover:scale-
105 transition-all flex items-center gap-2 font-black text-xs uppercase tracking-widest"><Plus
size={18}/> New Resource</button>

        </header>

        <div className="bg-white rounded-[3rem] border border-slate-100 shadow-sm overflow-hidden
min-h-[400px] flex items-center justify-center">

           <div className="text-center space-y-4">

              <Database size={48} className="text-slate-100 mx-auto" />

              <p className="text-slate-400 font-bold italic text-sm">Synchronizing with system
ontology...</p>

           </div>

        </div>

      </main>

    </div>

  );


  return (

    <div className="flex h-screen bg-white text-slate-900 overflow-hidden font-sans">

      <nav className="w-20 bg-slate-900 border-r border-slate-800 flex flex-col items-center py-
10 gap-10 z-50 shrink-0 shadow-2xl">

        <div className="w-12 h-12 bg-indigo-600 rounded-2xl flex items-center justify-center
text-white font-black text-2xl shadow-2xl shadow-indigo-600/30">S</div>

        <div className="flex flex-col gap-6">

          <button onClick={() => setView('browser')} className={`p-3.5 rounded-2xl transition-all
${view === 'browser' ? 'bg-indigo-600 text-white shadow-xl shadow-indigo-600/20' : 'text-slate-
500 hover:text-white hover:bg-slate-800'}`} title="Template Library"><FolderOpen
size={24}/></button>

          <button onClick={() => setView('studio')} className={`p-3.5 rounded-2xl transition-all
${view === 'studio' ? 'bg-indigo-600 text-white shadow-xl shadow-indigo-600/20' : 'text-slate-500
hover:text-white hover:bg-slate-800'}`} title="Template Studio"><Box size={24}/></button>

          <button onClick={() => setView('admin')} className={`p-3.5 rounded-2xl transition-all
${view === 'admin' ? 'bg-indigo-600 text-white shadow-xl shadow-indigo-600/20' : 'text-slate-500
hover:text-white hover:bg-slate-800'}`} title="System Governance"><Settings size={24}/></button>

        </div>

      </nav>


      <div className="flex-1 relative overflow-hidden bg-white">

        {view === 'browser' ? renderBrowser() : view === 'studio' ? renderStudio() :
renderAdmin()}
```

```
      </div>
    </div>
  );
};

export default App;
```